CSA4707-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 6: Gradient Descent, SGD, Momentum and Adam Comparison

Problem Statement: A logistics company trains a delivery-time prediction model. Training is slow, and the engineering lead asks which optimiser converges fastest and most reliably. A delivery dataset with distance, traffic index, weight and time of day as inputs and delivery time as the target. The objective is to implement and compare batch GD, SGD, Momentum and Adam. Use NumPy, TensorFlow, Keras, Matplotlib.

Tasks:

1. Implement batch GD, SGD and Momentum from scratch on a simple loss surface.
2. Plot each optimiser's path on a 2-D loss contour.
3. Train the same Keras regression model with SGD, SGD+Momentum and Adam.
4. Plot loss against epochs for all optimisers.
5. Vary the learning rate and note divergence or slow convergence.

FORMULA:

1.Gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

2.Stochastic gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla L_i(\theta_t)$$

3.Momentum

$$v_t = \beta v_{t-1} + \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

4.Mean squared error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras

np.random.seed(10)
tf.random.set_seed(10)

def loss(x, y):
    return x**2 + 4*y**2

def gradient(x, y):
    return np.array([2*x, 8*y])

def optimize(method, lr=0.1, steps=50):
    point = np.array([-4.0, 3.0])
    velocity = np.zeros(2)
    path = [point.copy()]

    for i in range(steps):
        if method == "SGD":
            grad = gradient(*point) + np.random.normal(0, 0.5, 2)
        else:
            grad = gradient(*point)
```

```
        if method == "Momentum":
            velocity = 0.9 * velocity + grad
            point -= lr * velocity
        else:
            point -= lr * grad

        path.append(point.copy())

    return np.array(path)

plt.figure(figsize=(8, 6))
x = np.linspace(-5, 5, 100)
y = np.linspace(-4, 4, 100)
X, Y = np.meshgrid(x, y)
plt.contour(X, Y, loss(X, Y), levels=25)

for method in ["GD", "SGD", "Momentum"]:
    path = optimize(method, lr=0.05)
    plt.plot(path[:, 0], path[:, 1], marker="o",
             markersize=2, label=method)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Optimiser Paths")
plt.legend()
plt.show()

X = np.random.rand(500, 4)
y = 5 + 10*X[:, 0] + 8*X[:, 1] + 3*X[:, 2] + 2*X[:, 3]
y += np.random.normal(0, 0.5, 500)
```

```
X_train, X_val = X[:400], X[400:]
y_train, y_val = y[:400], y[400:]

def build_model():
    model = keras.Sequential([
        keras.layers.Input(shape=(4,)),
        keras.layers.Dense(16, activation="relu"),
        keras.layers.Dense(1)
    ])
    model.compile(optimizer="sgd", loss="mse")
    return model

optimizers = {
    "SGD": keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ),
    "Adam": keras.optimizers.Adam(learning_rate=0.01)
}

histories = {}

for name, optimizer in optimizers.items():
    model = build_model()
    model.compile(optimizer=optimizer, loss="mse")
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=50, batch_size=32, verbose=0
    )
```

```
    histories[name] = history.history

    plt.plot(history.history["loss"], label=name)

plt.xlabel("Epochs")
plt.ylabel("Training Loss")
plt.title("Optimiser Comparison")
plt.legend()
plt.grid()
plt.show()

for lr in [0.001, 0.01, 0.1, 1.0]:
    model = build_model()
    model.compile(
        optimizer=keras.optimizers.SGD(learning_rate=lr),
        loss="mse"
    )
    history = model.fit(
        X_train, y_train, epochs=30,
        batch_size=32, verbose=0
    )
    plt.plot(history.history["loss"], label=f"LR={lr}")

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Learning Rate Comparison")
plt.legend()
plt.grid()
plt.show()
```

OUTPUT:

